

تحلیل و طراحی سیستم ها

پروژه درس
فاز دوم

اعضای تیم توسعه:
سینا محمدی
امیرحسین قاسمی پور
مهرشاد ولیزاد ارجمند
امیرمحمد شاهرضائی
محمد ارمیا قاصری
نیما نطقی

استادان درس:
دکتر محمدامین فضلی
محسن حسامالحکما

طراحان پروژه:
مانی ابراهیمی
محمد جعفری پور
علی محمدزاده شیستری

نیم سال بهار ۱۴۰۵



دانشکده مهندسی کامپیوتر

خلاصه اجرایی فاز دوم:

در این فاز، فاز ۱ را ادامه دادیم و طبق چیزی که در آنجا انجام داده بودیم یک اپلیکشین ساختیم. این محصول تمام نیازمندی های مذکور اعم از ایجاد امن حساب کاربری، احراز هویت مبتنی بر نشست (Session Authentication)، پروفایل کاربری قابل ویرایش، گفت و گو خصوصی، گروه های خصوصی، کانال ها موضوع (Topic) دار، پشتیبانی از نقش های مختلف در کانال، پیام های متنی و رسانه ای، ویرایش و حذف پیام، قابلیت جست و جو اختصاصی متن پیام ها در هر گفت و گو و همچنین اعلان های پایدار پشتیبانی میکند.

قابلیت های امتیازی نیز در این محصول پشتیبانی شده اند، اعلان ها توسط WebSocket های احراز هویت شده و بدون نیاز به تازه سازی دستی دریافت میشوند. همچنین کاربران قابلیت ارسال پیام زمابندی شده دارند و حتی اگر در آن زمان آفلاین باشند، این پیام توسط Celery در زمان مقرر ارسال میشود.

معماری پروژه همچنان یک Monolithic ماژولار (Modular Monolith) با چندین فرایند اجرایی مستقل باقی مانده است. این تصمیم طراحی سرویس محور و مبتنی بر کانتینر ارائه شده در فاز اول را حفظ میکند و از پیچیدگی های غیرضروری ناشی از میکروسرویس جلوگیری میکند.

تداوم فاز ۱ :

مفاهیم حفظ شده:

دامنه ای که پیاده سازی کردیم همچنان اصطلاحات و روابط پیاده سازی شده در فاز اول را حفظ کرده. مفاهیمی مثل:

- کاربر (User)
- فضای گفت و گو (ChatSpace)
- موضوع (Topic)
- نقش (Role)
- SpaceMembership در پیاده سازی، که همان User_ChatSpace است.
- پیام (Message)

رابط کاربری سه ستونه نیز مطابق با Wireframe ای که در فاز اول طراحی کردیم، پیاده سازی شده است و موارد زیر را شامل میشود:

- ستون سمت چپ برای نمایش گفت و گو های خصوصی، گروه ها و کانال ها.

- بخش مرکزی برای نمایش موضوع فعال، جست و جو، تاریخچه پیام ها، زمانبندی ارسال پیام و ویرایشگر نوشتن پیام.
- ستون سمت راست برای نمایش اعضا و نقش های آنها.
- پنجره (Modal) مخصوص مشاهده و ویرایش پروفایل کاربر.

توسعه های ضروری در پایگاه داده:

نمودار ER اولیه قادر به برطرف کردن بعضی از نیازمندی ها نبود و به همین دلیل موارد زیر اضافه شدند:

- افزودن نوع DIRECT به ChatSpace برای پشتیبانی از پیام های مستقیم بین دو کاربر.
- اضافه شدن مالک (Owner) و کلید های خارجی (Foreign Keys) مورد نیاز برای اعمال صریح روابط.
- افزودن تصویر پروفایل (Avatar)، بیوگرافی (Biography) و سیاست دعوت به گروه یا همان (Group Invite Policy)
- تعریف کامل قابلیت های نقش ها به جای وجود تنها یک فیلد برای قابلیت حذف پیام.
- افزودن وضعیت پیام، ارتباط پیام با Topic و زمان های ایجاد، ارسال، ویرایش و زمانبندی پیام.
- موجودیت Attachment برای نگهداری تصاویر، ویدئو، فایل های صوتی و سایر فایل ها.
- موجودیت Notification برای پشتیبانی از اعلان های پایدار و اعلان های بلادرنگ.

تمامی این تغییرات، در راستای برآورده سازی نیازمندی ها فاز ۱ را گسترش داده و نه جایگزینی ای انجام شده. توضیح کامل دلایل این تغییرات در فایل docs/phase2/architecture.md ارائه شده است.

مواردی که آگاهانه از محدوده پروژه خارج شده اند:

قابلیت ورود یکپارچه (SSO) و کنترل ها مربوط به کانال های صوتی (Voice Channels) تنها به عنوان مواردی کلی و اولیه در Wireframe پیشنهادی مطرح شده بودند، این قابلیت ها در نیازمندی های رسمی پروژه، User Story ها، نمودار ERD و همچنین جریان فناوری انتخاب شده وجود نداشتند بنابراین در محصول نهایی به عنوان ویژگی های ناقص یا ظاهری ارائه نشدند. مواردی مثل واکنش به پیام ها، تماس صوتی و تصویری، رسید خواندن پیام و درخواست دوستی نیز نه در نیازمندی های رسمی پروژه آمده اند و نه در درون محدوده این پروژه درسی قرار میگیرند برای همین حذف شدند.

معماری نهایی:

مرورگر (Browser) فقط از طریق یک نقطه ورودی Nginx به سامانه متصل میشود. وظایف Nginx به شرح زیر است:

- ارائه برنامه React
- هدایت درخواست های /api/ به Django
- ارتقای اتصال های /ws/ به همان سرویس ASGI جهت برقراری ارتباط WebSocket

PostgreSQL به عنوان منبع دائمی و اصلی داده ها عمل میکند.

Redis توسط Django channels برای توزیع بلادرنگ پیام ها استفاده میشود.

RabbitMQ وظیفه انتقال وظایف celery را به عهده دارد.

Celery Beat نیز فرآیند بررسی پیام های زمان بندی شده را آغاز می کند؛ به گونه ای که حتی پس از راه اندازی مجدد سیستم نیز این فرآیند بدون مشکل ادامه یابد.

سرویس های زمان اجرا (Runtime Services):

۱. دروازه Nginx: مسئول ارائه فایل های استاتیک سمت کاربر، هدایت درخواست های HTTP، ارتقای اتصال WebSocket، اعمال محدودیت روی درخواست ها و تنظیم Header های امنیتی.
۲. Django ASGI: ارائه دهنده موارد زیر است:

- REST API
- احراز هویت مبتنی بر نشست
- سرویس های دامنه (Domain Services)
- کنترل دسترسی فایل ها
- مصرف کننده های وب سوکت (WebSocket Consumers)

۳. PostgreSQL: نگهداری پایدار اطلاعات مربوط به موارد زیر:

- حساب های کاربری
- فضاهای گفت و گو
- عضویت ها
- نقش ها
- موضوعات (Topics)
- پیام ها
- فایل های پیوست
- اعلان ها

۴. Redis: تحویل سریع پیام ها به گروه های Django Channels با کمترین تاخیر ممکن.

۵. RabbitMQ: Broker پایدار برای Celery.

۶. Celery worker و Celery beat: مسئول ارسال پیام های زمانبندی شده با در نظر گرفتن سطح دسترسی کاربران.

تمامی این سرویس ها به صورت قابل بازتولید در فایل `compose.yaml` تعریف شده اند. همچنین برای اجرای آزمون های محلی بدون تغییر در کد محصول، میتوان از SQLite و یک لایه کانال درون حافظه ای (In-memory channel layer) استفاده کرد.

تحقق نیازمندی ها:

نیازمندی اول، ثبت نام، ورود و خروج: کاربران با استفاده از مواردی مثل نام کاربری یکتا، آدرس ایمیل یکتا، رمز عبور اعتبار سنجی شده ثبت نام میکنند. Django رمز عبور را هش کرده و یک نشست از نوع HttpOnly ایجاد میکند. ورود به سامانه از طریق ایمیل انجام می شود و در صورت نامعتبر بودن اطلاعات ورود، یک پاسخ عمومی و بدون افشای دلیل خطا نمایش داده می شود. فرآیند های ثبت نام و ورود و خروج توسط CSRF Protection محافظت شده اند و عملیات Log out یا خروج نشست کاربر را باطل میکند.

نیازمندی دوم، ارسال و دریافت پیام: اعضا میتوانند متن، یک یا چند فایل، یا ترکیبی از هر دو را ارسال کنند و تمامی فضا های گفت و گو مثل Direct، چنل و یا گروه از یک سرویس اعتبار سنجی مشترک برای ارسال پیام استفاده میکنند. تاریخچه پیام ها به ترتیب زمانی مرتب شده اند و به صورت صفحه بندی شده (Pagination) نمایش داده میشوند.

نیازمندی سوم، ویرایش و حذف پیام: تنها فرستنده پیام مجاز به ویرایش متن پیام ارسال شده است و پاسخ سامانه شامل برچسب یا زمان ویرایش خواهد بود. عملیات حذف پیام نیز توسط فرستنده پیام یا کاربرانی که مجوز مدیریتی لازمه را در اختیار دارند قابل انجام است. دقت کنید تمامی این درخواست ها حتی اگر کاربر به صورت دستی ایجادشان کرده باشد مجددا در سمت سرور مورد ارزیابی و اعتبار سنجی قرار میگیرند.

نیازمندی چهارم، کانال ها و Topic ها: سازنده یک کانال به صورت خودکار و به طور پیش فرض مالک (Owner) آن کانال محسوب میشود و تمام دسترسی ها و مجوز های مدیریتی لازم را دارد. از بعضی از فعالیت هایی که نیازمند مجوز هستند میتوان ایجاد تاپیک، ویرایش نام آن و حذف آن را نام برد. دقت کنید هر پیامی که در کانالی فرستاده میشود باید به یکی از تاپیک های همان کانال وابسته باشد.

نیازمندی پنجم، گروه ها: کاربران میتوانند یک گروه را با یک یا چند کاربر دیگر ایجاد کنند. اگر سیاست دعوت به گروه یک کاربر روی NOBODY تنظیم شده باشد هیچکس نمیتواند او را به گروهی

اضافه کند. همچنین در یک گروه اعضا میتوانند کاربر جدید اضافه کنند، گروه را ترک کنند و عملیات مدیریتی مربوط به گروه را مطابق متن صریح پروژه انجام دهند.

نیازمندی ششم، ویرایش و حذف کانال ها و گروه ها: مالک کانال یا هر نقش دیگری که مجوز `can_manage_space` را داشته باشد میتواند گروه را ویرایش یا حذف کند، همچنین تمام اعضای فعلی گروه توانایی ویرایش یا حذف آن را دارند. البته دقت کنید در پیاده سازی ما تمامی عملیات های مخرب قبل از اجرا نیازمند تایید کاربر در رابط کاربری است.

نیازمندی هفتم، رسانه ها (Media): هر پیام میتواند حداکثر ۵ فایل داشته باشد، اندازه هر فایل نیز حداکثر ۱۰ مگابایت است. تصویر ها، ویدئو ها، فایل های صوتی و اسناد از جمله دسته های پشتیبانی شده هستند. نام فایل ها و مسیر های ذخیره سازی آنها پاکسازی (Sanitize) میشوند و دانلود فایل های پیوست تنها برای اعضای فعلی همان فضای گفت و گو امکان پذیر است. همچنین ممکن است یک نقش در کانال بتواند ارسال پیام های چندرسانه ای را غیرفعال کند در حالی که همچنان امکان ارسال پیام متنی وجود دارد.

نیازمندی هشتم، نقش ها و محدودیت های دسترسی: نقش های کانال به صورت رکورد هایی از پایگاه داده ذخیره میشوند و دارای نام و مجموعه ای از قابلیت های قابل ویرایش هستند. این قابلیت ها شامل ارسال پیام، ارسال فایل های رسانه ای، مدیریت Topic ها، مدیریت اعضا، حذف پیام، مدیریت نقش ها و همچنین مدیریت فضای گفت و گو هستند. مالک به صورت ضمنی همه این قابلیت ها را دارد. سامانه از ایجاد یا اختصاص هر نقشی که باعث شود یک مدیر، اختیاری بیشتر از اختیارات خودش به شخص دیگری اعطا کند، جلوگیری می کند. به عبارت دیگر، مدیر تنها می تواند مجوزهایی را به دیگران بدهد که خودش نیز آن مجوزها را داشته باشد.

نیازمندی نهم، جست و جوی پیام ها: این جست و جو نسبت به حروف بزرگ و کوچک حساس نیست و تنها بر اساس متن پیام ها اتفاق می افتد. همچنین به فضای گفت و گوی انتخاب شده (ChatSpace) و به موضوع (Topic) در صورت وجود، محدود است. همچنین اطلاعات فقط به اعضای عضو همان فضای گفت و گو داده میشوند و هیچ اطلاعاتی به دیگر کاربران داده نمیشود.

نیازمندی دهم، پروفایل کاربران: هر کاربر میتواند اطلاعاتی اعم از نام کاربری، آدرس ایمیل، تصویر پروفایل، بیوگرافی و همچنین سیاست دعوت به گروه را ویرایش کند، همچنین کاربری احراز هویت شده میتواند نمایه دیگر کاربران را در یک پنجره (Modal) با قابلیت تنها خواندن، مشاهده کنند.

نیازمندی یازدهم، اعلان ها: رخداد های زیر باعث ایجاد اعلان های پایدار میشوند:

- دریافت پیام جدید

- رخدادهای مربوط به عضویت در کانال یا گروه.

همچنین کاربران میتوانند

- اعلان های خوانده نشده را مشاهده کنند.
- تمام اعلان ها را مشاهده کنند.
- یک اعلان را به عنوان خوانده شده علامت بزنند.
- تمامی اعلان ها را یکجا به عنوان خوانده شده علامت بزنند.

اعلان های پایدار این معنی را میدهند که اگر اتصال کاربر برای مدت کوتاهی قطع شود، هیچ رویدادی از دست نخواهد رفت.

نیازمندی های امتیازی:

پیام ها و اعلان های **بلادرنگ**: پس از آنکه یک درخواست HTTP با موفقیت در پایگاه داده ثبت (Commit) شود، Backend یک رویداد را در گروه مربوطه در Django Channels که مبتنی بر Redis است منتشر می کند. مصرف کننده های (Consumers) مربوط به هر فضای گفتگو هنگام برقراری اتصال، احراز هویت و عضویت کاربر را بررسی می کنند و این بررسی پیش از ارسال هر رویداد نیز مجدداً انجام می شود. یک اتصال WebSocket جداگانه و احراز هویت شده برای هر کاربر نیز مسئول انتقال اعلان ها و رویدادهای مربوط به عضویت است. کلاینت React در صورت نیاز، به طور خودکار اتصال را دوباره برقرار کرده و وضعیت پایدار (Durable State) را بازیابی می کند؛ بنابراین دیگر نیازی نیست کاربر برای مشاهده پیام ها یا اعلان های جدید، کل صفحه را با F5 یا Ctrl+R تازه سازی کند.

پیام های **زمانبندی شده**: رابط برنامه نویسی (API) پیام هایی را که قرار است در آینده ارسال شوند، با وضعیت PENDING و بر اساس زمان استاندارد جهانی (UTC) ذخیره می کند. هر پنج ثانیه یکبار، Celery Beat فرآیند بررسی پیام هایی را که زمان ارسالشان فرا رسیده آغاز می کند. سپس یکی از worker های celery:

- رکورد های مربوطه را قفل میکند.
- دوباره عضویت فرستنده را بررسی میکند.
- مجوز ارسال پیام، رسانه و Topic را مجدداً اعتبار سنجی میکند.
- پیام های معتبر را از وضعیت PENDING به SEND تغییر میدهد.
- اعلان های لازم را ایجاد میکند.
- تغییرات را در پایگاه داده ثبت (Commit) میکند.
- و در نهایت رویداد های بلادرنگ را منتشر میکند.

از آنجا که پیام های زمانبند شده در PostgreSQL ذخیره م شوند، حتی اگر مرورگر، Worker یا Broker از دسترس خارج شوند، این پیام ها از بین نخواهند رفت و هنگام زمان ارسال نیز نیازی نیست فرستنده آنلین باشد.

امنیت و قابلیت اطمینان (Security and Reliability):

سامانه برای افزایش امنیت و قابلیت اطمینان از مکانیزم های زیر استفاده می کند:

احراز هویت مبتنی بر Session و محدود به همان مبدأ (Same-Origin) برای REST و WebSocket ؛

اعتبارسنجی CSRF برای تمام درخواست های HTTP ناامن، از جمله ثبت نام و ورود؛

استفاده از الگوریتم های هش و اعتبارسنجی رمز عبور در Django ؛

بررسی سطح دسترسی اعضا و مجوزها در سطح هر شیء (Object-Level Permission Checks) ؛

اعتبارسنجی مبدأ WebSocket و بررسی مکرر عضویت کاربران؛

استفاده از مسیرهای ذخیره سازی تصادفی، بررسی نوع، پسوند و اندازه فایل ها و محافظت از دانلود فایل های پیوست؛

محدودسازی نرخ درخواست ها (Request Throttling) در Django REST Framework و Nginx ؛

مدیریت اطلاعات محرمانه و تنظیمات Cookie و HTTPS محیط تولید از طریق متغیرهای محیطی؛

نمایش محتوای پیام ها و پروفایل ها به صورت متن ساده (Plain Text Rendering) در React ؛

اعمال محدودیت های پایگاه داده برای تضمین یکتایی شناسه ها، عضویت ها، گفتگوهای مستقیم، نقش ها، Topic ها، Nonce ها و وضعیت های معتبر زمانبندی؛

انتشار رویدادهای بلادرنگ تنها پس از ثبت موفق تراکنش (Transaction-on-Commit) تا کاربران هرگز داده های Rollback شده را مشاهده نکنند؛

ارسال Idempotent پیام های زمان بندی شده همراه با بررسی مجدد مجوزها.

فهرست کامل این موارد در فایل docs/phase2/security.md قرار دارد.

اعتبارسنجی و تضمین کیفیت:

مخزن پروژه شامل آزمون‌های خودکار Backend برای موارد زیر است:

- احراز هویت، هش رمز عبور، جلوگیری از ثبت هویت تکراری، حریم خصوصی پروفایل و CSRF ؛
 - یکتایی گفت‌وگوهای مستقیم، گروه‌ها، کانال‌ها، عضویت‌ها، Topic ها و نقش‌ها؛
 - ماتریس کامل مجوزهای مربوط به ویرایش، حذف، رسانه و جستجوی پیام؛
 - دسترسی محافظت شده به فایل‌های پیوست و پاکسازی آن‌ها؛
 - اعلان‌های پایدار؛
 - رد اتصال WebSocket برای کاربران ناشناس یا غیرعضو و همچنین تحویل صحیح رویدادها؛
 - وظایف زمان‌بندی شده شامل حالت‌های: سررسیدشده (Due) ، مربوط به آینده (Future) ، لغوشده (Cancelled)، تکرار شده (Repeated) و لغوشدن مجوزها (Revoked Permission) ؛
 - Endpoint های بررسی زنده بودن (Liveness) و آمادگی سرویس (Readiness)
- علاوه بر این، آزمون‌های Frontend نیز مهم‌ترین جریان‌های مربوط به احراز هویت، مدیریت وضعیت برنامه و تعامل کاربران را پوشش می‌دهند.
- در خط لوله CI نیز موارد زیر اجرا می‌شوند:
- بررسی تنظیمات و Migration ها
 - Ruff
 - Pytest
 - ESLint
 - Vitest
 - ساخت نسخه Production توسط Vite
- همچنین اسکریپت‌های بررسی سلامت Docker و آزمون‌های اولیه (Smoke Tests) HTTP نیز در پروژه وجود دارند.

اجرای محصول:

برای اجرای پروژه موارد زیر انجام میشوند:

```
cp .env.example .env
```

```
docker compose up --build -d
```

```
docker compose exec backend python manage.py seed_demo
```

سپس برنامه از طریق آدرس زیر در دسترس خواهد بود:

<http://localhost:8080>

فایل README شامل اطلاعات حساب های نمایشی (Demo Credentials) و همچنین روش پیشنهادی برای نمایش پروژه، با استفاده از دو مرورگر است.

دستورات بررسی کیفیت:

```
make test
```

```
make check
```

```
make smoke
```

حوزه مشارکت اعضای تیم:

سینا محمدی – Product owner/Accounts:

- تعریف معیار های پذیرش
- ثبت نام کاربران
- احراز هویت
- مدیریت جریان های پروفایل
- رهگیری نیازمندی ها (Traceability)

محمد ارمیا قاصری – Scrum master/Spaces:

- گروه ها
- کانال ها
- عضویت ها
- تاپیک ها

- ثبت سوابق Iteration ها

مهرشاد ولیزاده ارجمند – Data and backend:

- طراحی شما
- Migration ها
- پیامها
- فایل های پیوست
- جست و جو
- اعلان ها
- داده ها نمایشی (Demo Data)

امیرمحمد شاهرضایی – Infrastructure:

- Docker
- Nginx
- ASGI
- Redis و WebSocket ها
- Celery و RabbitMQ
- ارسال پیام های زمانبندی شده

امیرحسین قاسمیپور – UI:

- طراحی سیستم رابط کاربری React
- رابط سه ستونه پیامرسان
- پنجره های محاوره ای (Dialogs)
- پیام ها
- رسانه
- جست و جو
- زمانبندی پیام
- رفتار واکنش گرا (Responsive)

نیما نطقی – Architecture and QA:

- طراحی مدل مجوز ها
- بازیابی یکپارچگی سیستم

- آزمون ها
- CI
- مستندات معماری
- گزارش نهایی انطباق با نیازمندی ها

مخزن و تخته پروژه (Repository and Project Board):

آدرس مخزن پروژه (Repository):

<https://github.com/Sina-rokna/SD-EchoMessenger>

آدرس تخته پروژه (Project Board):

<https://github.com/users/Sina-rokna/projects/2/views/1>

این مخزن علاوه بر نگهداری گزارش فاز اول، نمودارها و Wireframe های اولیه، شامل کد نهایی پروژه، Migration ها، آزمون ها، فایل های استقرار (Deployment Files) و مستندات مربوط به فاز دوم نیز هست.

پایان!

با تشکر از توجه شما!